

Team Project Final Report

As part of Database (420-231-VA) at Vanier College

Submitted to Nagat Drawel

Work by Mathieu Bernardin and Kiavash Emrani

Table of Contents

Introduction.....	3
Conceptual Design of the Database.....	3
Logical Database Schema.....	4
Data Definition Statements.....	4
Data Manipulation Statements.....	7
Database Management Package.....	9
Complex Queries.....	9
Views.....	9
Procedures.....	11
Trigger.....	13
Other Important SQL Statements.....	14
Execution Results.....	16
Concluding Remarks.....	18

Introduction

This project's goal was to design a database for a non-descript dentist office. Sample management tools, such as views and procedures and views, were also to be included in the database design. The database was to be designed using MS SQL Server, the language which we have studied in this course this semester. Additionally, this project was intended as a general review of all course material this semester, and a chance to practice out teamwork abilities.

Conceptual Design of the Database

After all changes to the database, below is the final entity relationship (ER) diagram of the database, with foreign keys excluded for brevity. An alternate version of this diagram is available in this project repository under /Deliverable3/finalErDiagramWithForeignKeys3NF.jpg for documentational purposes.

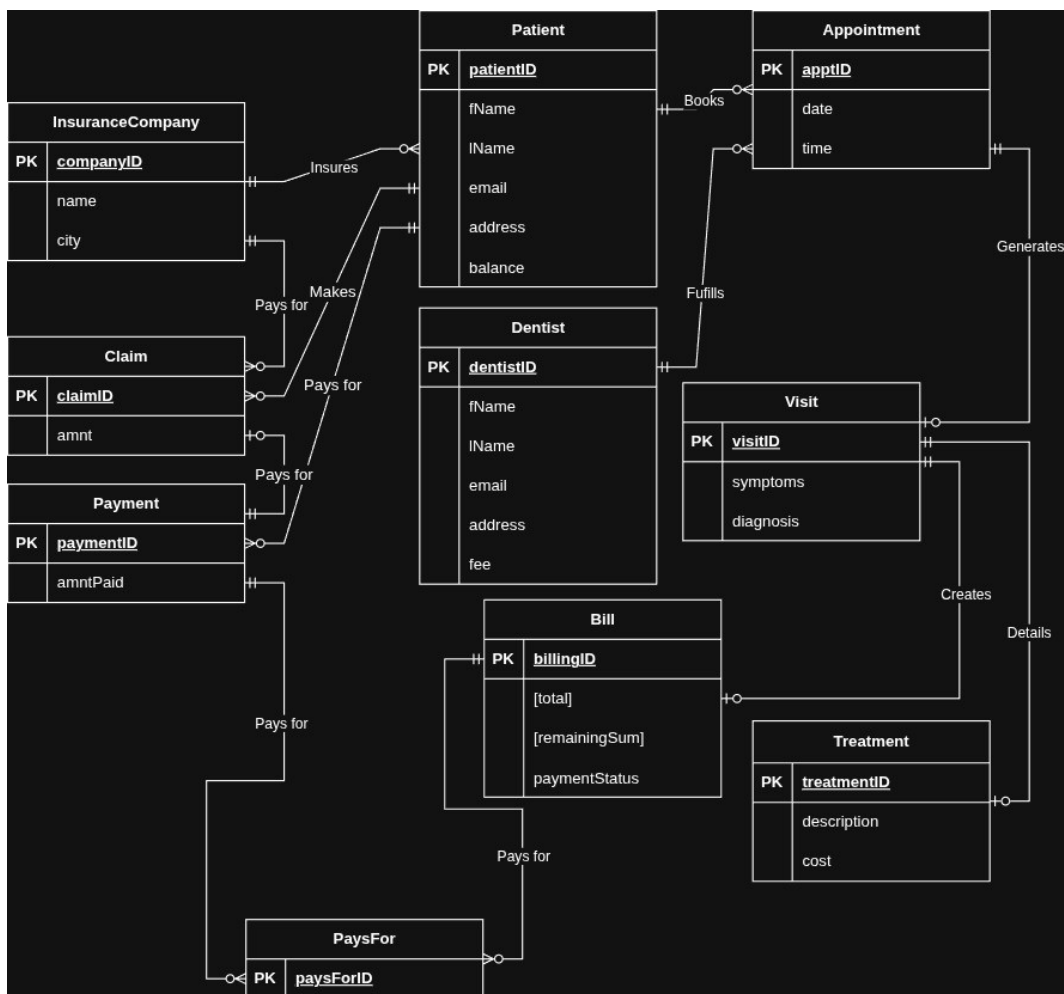


Figure 1: ER Diagram

Logical Database Schema

Following the changes to the ER diagram, below is the updated logical schema for this database.

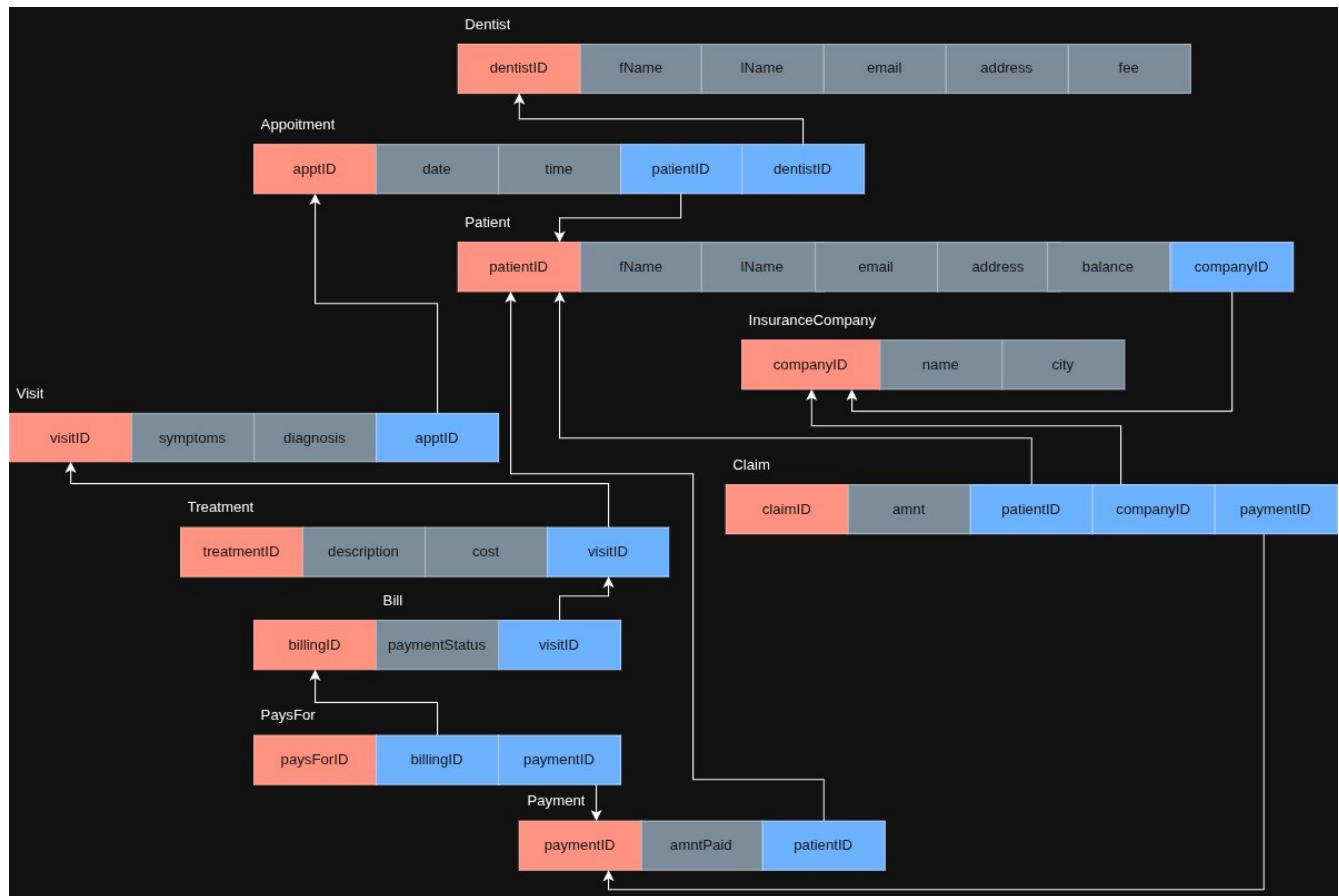


Figure 2: Logical Schema

Data Definition Statements

In order to define this schema within the database, the following SQL statements were written.

```
CREATE TABLE InsuranceCompany(
companyID INT CONSTRAINT PK_InsuranceCompany PRIMARY KEY,
name VARCHAR(30),
city VARCHAR(30)
);
CREATE TABLE Patient(
patientID INT IDENTITY CONSTRAINT PK_Patient PRIMARY KEY,
fName VARCHAR(20),
lName VARCHAR(20),
email VARCHAR(40) CONSTRAINT Patient_email_unique UNIQUE,
address VARCHAR(40)
);
```

```

CREATE TABLE Dentist(
dentistID INT IDENTITY CONSTRAINT PK_Dentist PRIMARY KEY,
fName VARCHAR(20),
lName VARCHAR(20),
email VARCHAR(40) CONSTRAINT Dentist_email_unique UNIQUE,
address VARCHAR(40),
fee MONEY CONSTRAINT Dentist_fee_CheckPositive CHECK(fee>=0)
);

CREATE TABLE Appointment(
apptID INT CONSTRAINT PK_Appointment PRIMARY KEY,
patientID INT CONSTRAINT Appointment_Patient_FK FOREIGN KEY REFERENCES Patient ON DELETE
CASCADE CONSTRAINT Appointment_Patient_NotNull NOT NULL, -- on delete cascade as if a patient
is removed from the system we no longer need to worry about their appointments
dentistID INT CONSTRAINT Appointment_Dentist_FK FOREIGN KEY REFERENCES Dentist CONSTRAINT
Appointment_Dentist_NotNull NOT NULL, -- on delete cascade as if a doctor is removed from the
system we no longer need to worry about their appointments
date DATE,
time TIME
);

CREATE TABLE Visit(
visitID INT CONSTRAINT PK_Visit PRIMARY KEY,
apptID INT CONSTRAINT Visit_Appointment_FK FOREIGN KEY REFERENCES Appointment ON DELETE
SET NULL, -- on delete set null as if an appointment is removed we still need to keep visit details for
financial records
symptoms VARCHAR(150),
diagnosis VARCHAR(30)
);

CREATE TABLE Treatment(
treatmentID INT CONSTRAINT PK_Treatment PRIMARY KEY,
visitID INT CONSTRAINT Treatment_Visit_FK FOREIGN KEY REFERENCES Visit ON DELETE CASCADE
CONSTRAINT Treatment_Visit_NotNull NOT NULL, -- on delete cascade as if we're removing a visit we
must no longer care about the financial records
description VARCHAR(40),
cost MONEY
);

```

```

CREATE TABLE Bill(
billID INT CONSTRAINT PK_Bill PRIMARY KEY,
visitID INT CONSTRAINT Bill_Visit_FK FOREIGN KEY REFERENCES Visit ON DELETE CASCADE
CONSTRAINT Bill_Visit_NotNull NOT NULL, -- on delete cascade as if we're removing a visit we must
no longer care about the financial records
paymentStatus CHAR(7)
);

CREATE TABLE Payment(
paymentID INT CONSTRAINT PK_Payment PRIMARY KEY,
patientID INT CONSTRAINT Payment_Patient_FK FOREIGN KEY REFERENCES Patient ON DELETE SET
NULL, -- on delete set null as if an patient is removed we still need to keep payment details for
financial records
amntPaid MONEY
);

CREATE TABLE Claim(
claimID INT CONSTRAINT PK_Claim PRIMARY KEY,
companyID INT CONSTRAINT Claim_InsuranceCompany_FK FOREIGN KEY REFERENCES
InsuranceCompany ON DELETE SET NULL, -- on delete set null as if an insurance company is removed
we still need to keep claim details for financial records
patientID INT CONSTRAINT Claim_Patient_FK FOREIGN KEY REFERENCES Patient ON DELETE SET
NULL, -- on delete set null as if an patient is removed we still need to keep claim details for financial
records
paymentID INT CONSTRAINT Claim_Payment_FK FOREIGN KEY REFERENCES Payment ON DELETE
CASCADE CONSTRAINT Claim_Payment_NotNull NOT NULL, -- on delete cascade as if a payment is
removed we must no longer care about the financial records
amnt MONEY
);

CREATE TABLE PaysFor(
paysForID INT CONSTRAINT PK_PaysFor PRIMARY KEY,
billID INT CONSTRAINT PaysFor_Bill_FK FOREIGN KEY REFERENCES Bill ON DELETE CASCADE
CONSTRAINT PaysFor_Bill_NotNull NOT NULL, -- on delete cascade as if a bill is removed we must no
longer care about the financial records
paymentID INT CONSTRAINT PaysFor_Payment_FK FOREIGN KEY REFERENCES Payment ON DELETE
CASCADE CONSTRAINT PaysFor_Payment_NotNull NOT NULL -- on delete cascade as if a payment is
removed we must no longer care about the financial records
);

ALTER TABLE Patient
ADD balance MONEY;

ALTER TABLE Treatment
ADD CONSTRAINT Treatment_cost_Check CHECK(cost>=0);

```

Data Manipulation Statements

Additionally, sample data for the database was required for this project; The following SQL statements were used to insert this data.

```
CREATE SEQUENCE treatmentSequence AS INT
NO CACHE;

INSERT INTO InsuranceCompany VALUES (1, 'SunLife Dental', 'Montreal'),
(2, 'BlueCross Health', 'Toronto'),
(3, 'Maple Insurance', 'Ottawa'),
(4, 'Northern Benefits', 'Calgary');

INSERT INTO Patient VALUES ('John', 'Smith', 'john.smith@email.com', '12 King St'),
('Emily', 'Johnson', 'emily.j@email.com', '55 Pine Ave'),
('Michael', 'Brown', 'michael.b@email.com', '9 Maple Rd'),
('Sarah', 'Davis', 'sarah.d@email.com', '101 Oak Lane'),
('Daniel', 'Wilson', 'dan.w@email.com', '88 Cedar Blvd');

INSERT INTO Dentist VALUES ('Alice', 'Martin', 'alice.martin@clinic.com', '200 Dental Ave', 120),
('Robert', 'Lee', 'robert.lee@clinic.com', '15 Queen St', 150),
('Sophia', 'Taylor', 'sophia.t@clinic.com', '33 Smile Rd', 180);

INSERT INTO Appointment VALUES (1, 1, 1, '2026-05-01', '09:00'),
(2, 2, 2, '2026-05-01', '10:30'),
(3, 3, 1, '2026-05-02', '11:00'),
(4, 4, 3, '2026-05-02', '13:00'),
(5, 5, 2, '2026-05-03', '15:30');

INSERT INTO Visit VALUES(1, 1, 'Tooth pain and swelling', 'Cavity'),
(2, 2, 'Bleeding gums', 'Gingivitis'),
(3, 3, 'Routine cleaning', 'Healthy'),
(4, 4, 'Jaw pain', 'Impacted wisdom tooth'),
(5, 5, 'Sensitive teeth', 'Enamel erosion');

INSERT INTO Treatment VALUES(NEXT VALUE FOR treatmentSequence, 1, 'Cavity filling', 250),
(NEXT VALUE FOR treatmentSequence, 2, 'Deep cleaning', 180),
(NEXT VALUE FOR treatmentSequence, 3, 'Dental cleaning', 100),
(NEXT VALUE FOR treatmentSequence, 4, 'Wisdom tooth extraction', 600),
(NEXT VALUE FOR treatmentSequence, 5, 'Fluoride treatment', 120);
```

```
INSERT INTO Bill VALUES (1, 1, 'Paid'),
(2, 2, 'Pending'),
(3, 3, 'Pending'),
(4, 4, 'Pending'),
(5, 5, 'Pending');

INSERT INTO Payment VALUES (1, 1, 370),
(2, 2, 180),
(3, 3, 100),
(4, 4, 600),
(5, 5, 120);

INSERT INTO Claim VALUES (1, 1, 1, 1, 200),
(2, 2, 2, 2, 140),
(3, 3, 3, 3, 80),
(4, 1, 4, 4, 500),
(5, 4, 5, 5, 90);

INSERT INTO PaysFor VALUES (1, 1, 1),
(2, 2, 2),
(3, 3, 3),
(4, 4, 4),
(5, 5, 5);
```

Database Management Package

As mentioned previously, a sample management package was included to review other SQL skills. Below are the SQL statements for each of these concepts.

Complex Queries

This first query is interesting as it allows an employee such as a receptionist to see all appointments which are scheduled for today, ordered by their time of day for ease of use.

```
SELECT CONCAT(P.fName, ' ', P.lName) 'Patient', CONCAT(D.fName, ' ', D.lName) 'Dentist', time 'Time of day'
FROM Appointment A JOIN Dentist D
ON A.dentistID=D.dentistID JOIN Patient P
ON A.dentistID=P.patientID
WHERE date=GETDATE()
ORDER BY time;
```

This second query is interesting as it allows average visit costs per dentist to be analyzed by a relevant staff member. This allows the owner's of the office to see who is their biggest revenue maker, or who may be a good budget option for cost-sensitive patients.

```
SELECT CONCAT(fName, ' ', lName) 'Dentist name', AVG(dbo.getTotal(billID)) 'Average' FROM Bill B
JOIN Dentist D
ON D.dentistID IN (SELECT dentistID FROM Appointment WHERE apptID IN (
SELECT apptID FROM Visit WHERE visitID=B.visitID))
GROUP BY dentistID, fName, lName;
```

Views

This first view allows the status of bills to be seen independently of any other information. This is helpful as it isolates only the information that someone who billing information is useful to needs to know. This improves security by not revealing other sensitive, personal information to such an employee.

```
CREATE VIEW billStatusView AS
SELECT dbo.getPayer(billID) 'Billed to',
       dbo.getTotal(billID) 'Bill total',
       dbo.getPaid(billID) 'Amount paid',
       paymentStatus 'Status'
FROM Bill;
```

This second view is essentially a more detailed version of the billStatusView. It allows accounting staff to see exact billing details for each outstanding bill, while hiding sensitive patient medical info. This more detailed version is necessary as billStatusView is designed more for an employee filling a receptionist-type role, who only needs to know what patients have bills to pay and how much they still have to pay, so that they can inform these patients. However, accounting staff must know more statistical information on these patients. Although these employees only need to analyze bills which must still be paid.

```
CREATE VIEW advancedBillingView AS
SELECT dbo.getPayer(billID)'Payer',
       dbo.getTotal(billID)'Bill total',
       dbo.getPaid(billID)'Amount paid',
       (SELECT COUNT(*)
        FROM Payment
        WHERE paymentID IN (
            SELECT paymentID
            FROM PaysFor P
            WHERE P.billID=B.billID))'Installments paid',
       (((SELECT SUM(amntPaid)
          FROM Payment P
          WHERE paymentID IN (
              SELECT paymentID
              FROM PaysFor
              WHERE billID=B.billID)))/(dbo.getTotal(B.billID))*100)'Percentage paid',
       (((SELECT SUM(amnt)
          FROM Claim C
          WHERE C.paymentID IN (
              SELECT paymentID
              FROM PaysFor P
              WHERE P.billID=B.billID)))/(dbo.getTotal(B.billID))*100)'Percentage claimed',
       (SELECT name
        FROM InsuranceCompany I
        WHERE I.companyID IN (
            SELECT companyID
            FROM Claim C
            WHERE C.paymentID IN (
                SELECT paymentID
                FROM PaysFor P
                WHERE P.billID=B.billID)))'Insurer'
FROM Bill B
WHERE paymentStatus='Pending';
```

Procedures

This first procedure ensures new patients being inserted follow proper conventions for certain attributes. These include names being formed solely of letters of the English alphabet and emails following the name@domain.extension format. If a rule is violated, the procedure raises an error which is then caught to set the success output variable to false (1). Then, if this variable is false, the procedure prints an error message; Otherwise, it inserts the provided information into the patient table, prints a success message, and sets the success output variable to true (0).

```
CREATE PROCEDURE standardNewPatient @fName VARCHAR(20),
                                     @lName VARCHAR(20),
                                     @email VARCHAR(40),
                                     @address VARCHAR(40),
                                     @success TINYINT OUTPUT AS
BEGIN
    BEGIN TRY
        IF NOT(@fName LIKE '%[a-z]%' OR @fName LIKE '%[A-Z]%')
            RAISERROR('Invalid information provided',16,1);
        IF NOT(@lName LIKE '%[a-z]%' OR @lName LIKE '%[A-Z]%')
            RAISERROR('Invalid information provided',16,1);
        IF NOT(@email LIKE '%@%.%')RAISERROR('Invalid information provided',16,1);
    END TRY
    BEGIN CATCH
        SET @success=1;
    END CATCH;
    IF(@success=1)PRINT 'Could not add patient; Invalid information was provided.';
    ELSE
    BEGIN
        INSERT INTO Patient(fName,lName,email,address)
            VALUES (@fName,@lName,@email,@address);
        PRINT 'New patient successfully added';
        SET @success=0;
    END;
END;
```

This second procedure allows users to rapidly update a single bill's payment status. It does this by first getting the amount that must be paid on this bill and the amount that has already been paid off through user-defined functions. If either of these values are -1 (i.e. the provided bill does not exist), an error is raised; Otherwise, it compares these two values. If the amount already paid is higher than the total to be paid, it sets the bill's payment status to paid; Otherwise, it sets it pending.

```
CREATE PROCEDURE updateBillStatus @billID INT AS
BEGIN
    DECLARE @total MONEY=(dbo.getTotal(@billID)),
            @amntPaid MONEY=(dbo.getPaid(@billID));
    IF(@total=-1 OR @amntPaid=-1)
        RAISERROR('No such bill to update...',16,1);
    ELSE IF(@amntPaid>=@total)
    BEGIN
        UPDATE Bill SET paymentStatus='Paid' WHERE billID=@billID;
        PRINT 'Bill status updated; Bill is paid off.';
    END;
    ELSE
    BEGIN
        UPDATE Bill SET paymentStatus='Pending' WHERE billID=@billID;
        PRINT 'Bill status not updated; Bill still has outstanding balance to be paid.';
    END;
END;
```

Trigger

As part of the project guidelines, a trigger to be written was detailed; This trigger follows these guidelines. It first inserts the data from the actual insert, then ensures the patient actually exists and the bill has a valid amount. If this verification fails, it rolls back the insertion; Otherwise, it updates the balance of the relevant patient.

```
CREATE TRIGGER autoBillPatient ON Bill INSTEAD OF INSERT AS
BEGIN
    BEGIN TRANSACTION insertion
        INSERT INTO Bill SELECT * FROM inserted;
    COMMIT;
    DECLARE @patientID INT=(SELECT patientID FROM Patient WHERE patientID IN (
        SELECT patientID FROM Appointment WHERE apptID IN (
            SELECT apptID FROM Visit WHERE visitID IN (
                SELECT visitID FROM inserted)))),
        @amount MONEY=dbo.getTotal((SELECT billID FROM inserted)),
        @success TINYINT=0;
    IF(@patientID IS NULL)
    BEGIN
        RAISERROR('Error billing patient; No such patient',16,1);
        SET @success=1;
    END
    IF(@amount=-1 OR @amount IS NULL)
    BEGIN
        RAISERROR('Error billing patient; Improper amount.',16,1);
        SET @success=1;
    END;
    IF(@success=1)
        ROLLBACK insertion;
    ELSE
        UPDATE Patient SET balance=balance+@amount;
END;
```

Other Important SQL Statements

In order to ensure functionality, three other functions were needed; They are described below.

This first function returns the total amount to be paid for a single specified bill. If the bill is not found it instead returns -1.

```
CREATE FUNCTION getTotal(@billID INT) RETURNS MONEY AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM Bill WHERE billID=@billID)
        RETURN -1;
    DECLARE @dentistsFee MONEY,
            @treatmentFee MONEY,
            @visitID INT=(SELECT visitID
                           FROM Bill
                           WHERE billID=@billID);
    DECLARE @apptID INT=(SELECT apptID
                           FROM Visit
                           WHERE visitID=@visitID),
            @treatmentID INT=(SELECT treatmentID
                               FROM Treatment
                               WHERE visitID=@visitID);
    DECLARE @dentistID INT=(SELECT dentistID
                              FROM Appointment
                              WHERE apptID=@apptID);
    SET @dentistsFee=(SELECT fee
                       FROM Dentist
                       WHERE dentistID=@dentistID);
    SET @treatmentFee=(SELECT cost
                        FROM Treatment
                        WHERE treatmentID=@treatmentID);
    IF(@treatmentFee IS NULL)
        SET @treatmentFee=0;
    IF(@dentistsFee IS NULL)
        SET @dentistsFee=0;
    RETURN @treatmentFee+@dentistsFee;
END;
```

This second function returns the total amount already paid for a single specified bill. If the bill is not found it instead returns -1.

```
CREATE FUNCTION getPaid(@billID INT) RETURNS MONEY AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM Bill WHERE billID=@billID)
        RETURN -1;
    DECLARE @amntPaid MONEY,
            @total MONEY=0;
    DECLARE paymentCursor CURSOR FOR
        SELECT amntPaid
        FROM Payment
        WHERE paymentID IN (SELECT paymentID
                           FROM PaysFor
                           WHERE billID=@billID);

    OPEN paymentCursor;
    FETCH NEXT FROM paymentCursor INTO @amntPaid;
    WHILE(@@FETCH_STATUS=0)
    BEGIN
        IF(@amntPaid IS NULL)
            SET @amntPaid=0;
        SET @total=@total+@amntPaid;
        FETCH NEXT FROM paymentCursor INTO @amntPaid;
    END;
    RETURN @total;
END;
```

This third function returns the name of the patient responsible for paying for a single specified bill. If the bill is not found it instead returns 'No such bill'

```
CREATE FUNCTION getPayer(@billID INT) RETURNS VARCHAR(41) AS
BEGIN
    IF NOT EXISTS(SELECT 1
        FROM Bill
        WHERE billID=@billID)
        RETURN 'No such bill';
    DECLARE @visitID INT=(SELECT visitID
        FROM Bill
        WHERE billID=@billID);
    DECLARE @apptID INT=(SELECT apptID
        FROM Visit
        WHERE visitID=@visitID);
    DECLARE @patientID INT=(SELECT patientID
        FROM Appointment
        WHERE apptID=@apptID);
    DECLARE @name VARCHAR(40)=(SELECT CONCAT(fName,' ',lName)
        FROM Patient
        WHERE patientID=@patientID);

    RETURN @name;
END;
```

Execution Results

Below are the results from executing the SELECT statements, or selecting everything from each of the views.

Results (1)		Messages	
Patient	↕ 🔍	Dentist	↕ 🔍
Time of d...		↕ 🔍	

Figure 3: Results for first complex query

Results (1)		Messages
	Dentist name	Average
1	Alice Martin	295.00
2	Robert Lee	300.00
3	Sophia Taylor	780.00

Figure 4: Results for second complex query

Results (1)		Messages										
	Billed to	↑↓	🔽	Bill total	↑↓	🔽	Amount Paid	↑↓	🔽	Status	↑↓	🔽
1	John Smith			370.00			370.00			Paid		
2	Emily Johnson			330.00			180.00			Pending		
3	Michael Brown			220.00			100.00			Pending		
4	Sarah Davis			780.00			600.00			Pending		
5	Daniel Wilson			270.00			120.00			Pending		

Figure 5: Results for *SELECT * FROM billStatusView*

Results (1)		Messages					
	Payer	↕ ▾ Bill total	↕ ▾ Amount paid	↕ ▾ Installments paid	↕ ▾ Percentage paid	↕ ▾ Percentage claimed	↕ ▾ Insurer
1	Emily Johnson	330.00	180.00	1	54.54	42.42	BlueCross Health
2	Michael Brown	220.00	100.00	1	45.45	36.36	Maple Insurance
3	Sarah Davis	780.00	600.00	1	76.92	64.10	SunLife Dental
4	Daniel Wilson	270.00	120.00	1	44.44	33.33	Northern Benefits

Figure 6: Results for *SELECT * FROM advancedBillingView*

Concluding Remarks

Throughout this project, proper coordination with teammates was an extreme challenge. As of writing this, almost three days past the deadline, I, Mathieu Bernardin, am still the sole contributor to this project. My first teammate, Jeet, dropped the course, as well as every other course, without prior warning and without further communication, already leaving us at a bit of a organizational disadvantage from the start. However, the real challenge was getting Kiavash to contribute anything. I have thoroughly communicated with him through every deliverable of this project, generally leaving him ample time to at least start his work. For this final deliverable, his assigned work was to write a single complex query, to detail the backup schema, and to define the user roles and other administrative portions of this deliverable. Meanwhile, I have completed all of his incompleted assigned work for the other two deliverables, while also completing every other part of this deliverable. If need be, I will complete his portion of this deliverable, but not for at least a week as I now desperately need a break after my end of semester.

Aside from this headache, I thoroughly enjoyed this project and I'm very proud of what I've accomplished in it, especially considering the circumstances. This sentiment extends to this course as a whole. Despite my not-quite-amazing attendance, I very much enjoyed the course and, although there are one or two concepts I have not yet fully grasped, I feel adequately prepared for the rest of my studies at Vanier involving databases and look forward to learning more.

That all being said, I'm going to bed now as I'm exceedingly tired. Thank you, Nagat, for teaching us all this semester, your work has been invaluable to my success. Good luck with all your correction and have a great summer.

Mathieu Bernardin